

Documentação Técnica do Sistema

JPL – GESTÃO DE E-COMMERCE DE MODA

1. VISÃO GERAL DA ARQUITETURA

O sistema **JPL** é uma aplicação web full-stack desenvolvida para o gerenciamento de um e-commerce de vestuário e acessórios. Sua estrutura é dividida de forma modular em três camadas principais:

Back-end Desenvolvido em Python utilizando o framework micro-web Flask para controle de rotas e processamento de solicitações.	Banco de Dados PostgreSQL , responsável pela persistência e pelas regras de negócio encapsuladas em procedimentos, funções e visões.	Front-end Interface renderizada com Jinja2 e estilizada responsivamente através da biblioteca Bootstrap 5 .
--	---	---

2. CAMADA DE BANCO DE DADOS

O script de banco de dados (`BD.sql`) estrutura a persistência do sistema através de 5 tabelas relacionais, além de automações transacionais via PL/pgSQL.

2.1. Estrutura de Tabelas

Tabela	Campos	Descrição & Regras
<code>categorias</code>	<code>id</code> (PK), <code>nome</code>	Armazena as classificações dos produtos (ex: Camisetas, Calças, Acessórios).
<code>produtos</code>	<code>id</code> (PK), <code>nome</code> , <code>preco</code> , <code>estoque</code> , <code>categoria_id</code> (FK)	Contém o inventário completo da loja com associação à tabela de categorias.
<code>clientes</code>	<code>id</code> (PK), <code>nome</code> , <code>email</code>	Guarda os dados cadastrais dos clientes. O campo <code>email</code> possui restrição de unicidade (UNIQUE).
<code>vendas</code>	<code>id</code> (PK), <code>cliente_id</code> (FK), <code>data_venda</code> , <code>valor_total</code>	Registra o cabeçalho das transações. <code>data_venda</code> possui valor padrão <code>CURRENT_TIMESTAMP</code> .
<code>itens_venda</code>	<code>id</code> (PK), <code>venda_id</code> (FK), <code>produto_id</code> (FK), <code>quantidade</code> , <code>preco_unitario</code>	Detalha os itens da venda. Possui exclusão em cascata (ON DELETE CASCADE) para <code>venda_id</code> .

2.2. Visão, Função e Procedure

View: `vw_relatorio_vendas`

Realiza a junção (`JOIN`) entre `vendas`, `clientes`, `itens_venda` e `produtos`. Retorna dados consolidados (ID da venda, nome/email do cliente, produto, quantidade, data e valor total) para renderização direta dos relatórios.

Function: `fn_calcular_subtotal`

Recebe `p_quantidade` INT e `p_preco` NUMERIC, retornando o produto matemático para compor o valor parcial dos itens no carrinho ou pedido.

3. CAMADA DE APLICAÇÃO (BACK-END)

A lógica do servidor está centralizada no arquivo `app.py` e utiliza o módulo dedicado de conexão `database.py`.

3.1. Configuração de Conexão (`database.py`)

Estabelece a comunicação com o SGBD PostgreSQL utilizando a biblioteca `psycopg2`. A conexão é parametrizada via dicionário `DB_CONFIG` contendo o nome do banco de dados (`Loja_Roupas`), usuário, senha e porta padrão (`5432`).

3.2. Rotas e Controladores (`app.py`)

- **Rota Principal (`/` – Função `index`):**
Abre a conexão com o banco e consulta a view `vw_relatorio_vendas` em ordem decrescente pela data (`ORDER BY data_venda DESC`). Renderiza o template `index.html` exibindo as vendas.
- **Rota de Produtos (`/produtos` – Função `produtos`):**
Consulta o catálogo unindo a tabela `produtos` à tabela `categorias`. Renderiza o template `produtos.html` com informações de inventário e preços.
- **Rota de Nova Venda (`/venda/nova` – Função `nova_venda`):**
GET: Busca todos os clientes e produtos com estoque superior a zero para popular as opções do formulário.
POST: Recebe os campos do formulário (`cliente_id`, `produto_id`, `quantidade`), executa a procedure `CALL pr_realizar_venda(...)`, efetua o `conn.commit()` e redireciona o usuário exibindo mensagens de status.

4. INTERFACE (FRONT-END)

Os templates utilizam a sintaxe do motor ****Jinja2**** e o framework ****Bootstrap 5**** para estilização visual e responsividade:

Template	Função e Componentes Visualizados
<code>base.html</code>	Layout base compartilhado. Contém a barra de navegação superior (<code>navbar</code>), links de navegação rápida, estrutura para mensagens de alerta (<code>flash messages</code>) e bloco <code>{% block content %}</code> .
<code>index.html</code>	Relatório geral de vendas apresentando tabela detalhada com ID, cliente, e-mail, produto, quantidade destacada, data/hora formatada e valor total em Reais (R\$).
<code>produtos.html</code>	Catálogo completo com badges dinâmicos para controle visual de estoque: VERDE (Normal), AMARELO (Baixo) e VERMELHO (Esgotado).
<code>nova_venda.html</code>	Formulário interativo para registro de ordens de venda com seletores para clientes, produtos disponíveis e campo de entrada numérica de quantidade.

5. DEPENDÊNCIAS DO PROJETO (`REQUIREMENTS.TXT`)

O ambiente de execução requer as seguintes bibliotecas Python instaladas:

```
Flask==3.0.2
psycopg2-binary==2.9.9
```